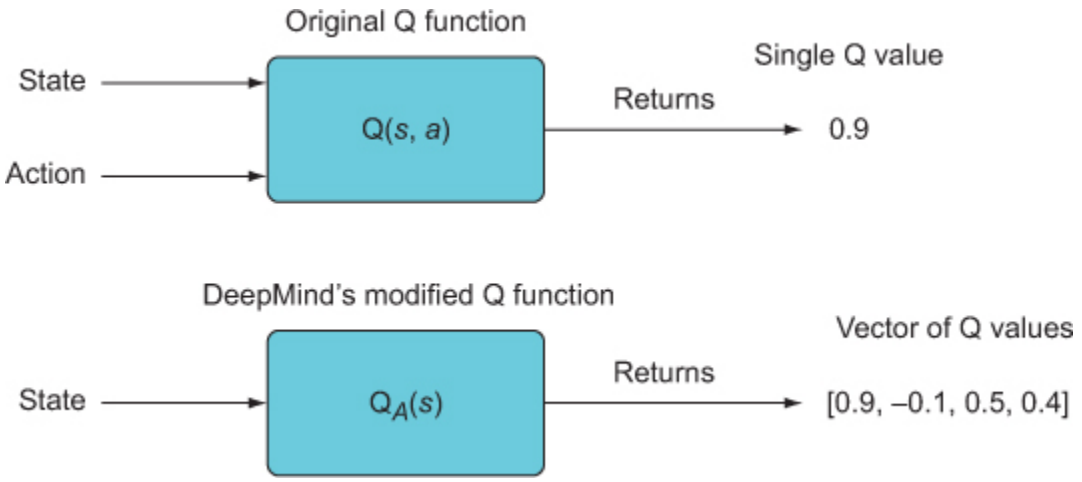
computes the value of some state, given that you're following a particular policy, π.

Figure 3.5. The original Q function accepts a state-action pair and returns the value of that state-action pair—a single number. DeepMind used a modified vector-valued Q function that accepts a state and returns a vector of state-action values, one for each possible action given the input state. The vector-valued Q function is more efficient, since you only need to compute the function once for all the actions.



Generally, we want to use the Q function because it can tell us the value of taking an action in some state, so we can take the action that has the highest predicted value. But it would be rather wasteful to separately compute the Q values for every possible action given the state, even though the Q function was originally defined that way. A much more efficient procedure, and the one that DeepMind employed in its implementation of deep Q-learning, is to instead recast the Q function as a vector-valued function, meaning that instead of computing and returning a single Q value for a single state-action pair, it will compute the Q values for all actions, given some state, and return the vector of all those Q values. So we might represent this new version of the Q function as $Q_A(s)$, where the subscript A denotes the set of all possible actions (figure 3.5).

Now it's easy to employ a neural network as our $Q_A(s)$ version of the Q function; the last layer will simply produce an output vector of Q values—one for each possible action. In the case of Gridworld, there are only four possible actions (up, down, left, right) so the output layer will produce 4-dimensional vectors. We can then directly use the output of the neural